

```
# Import the foundational libraries
import pandas as pd
import numpy as np

# Load your dataset into a pandas DataFrame (think of this as a virtual Excel s
df = pd.read_csv('dataset.csv')

# Success message so we know it worked
print("Data loaded successfully!")
```

Data loaded successfully!

Step 1: Loading the Data. Here we import pandas and load the student dropout dataset.

```
# Look at the first 5 rows of the data
display(df.head())
```

Daytime/evening attendance	Previous qualification	Nac
1	1	
1	1	
1	1	
1	1	
0	1	

Step 2: Data Inspection An AI needs clean data to function. We ran diagnostic checks to see the health of the dataset.

Viewing the Data: Using `df.head()`, we looked at the first five rows to understand what features we were working with (e.g., grades, tuition status, demographics).

Checking for Errors: Using `df.info()`, we confirmed two crucial things:

There were 4,424 rows and 0 missing values (meaning no data cleanup was required for blank cells).

Almost all columns were already formatted as numbers (int64 or float64), which is exactly what machine learning algorithms require.

```
# Check data types and look for missing (null) values
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4424 entries, 0 to 4423
Data columns (total 35 columns):
#   Column                                                                 Non-Null Count  Dtype
---  -
0   Marital status                                                         4424 non-null  int64
1   Application mode                                                         4424 non-null  int64
2   Application order                                                         4424 non-null  int64
3   Course                                                                  4424 non-null  int64
4   Daytime/evening attendance                                              4424 non-null  int64
5   Previous qualification                                                  4424 non-null  int64
6   Nacionality                                                            4424 non-null  int64
7   Mother's qualification                                                  4424 non-null  int64
8   Father's qualification                                                  4424 non-null  int64
9   Mother's occupation                                                     4424 non-null  int64
10  Father's occupation                                                     4424 non-null  int64
11  Displaced                                                              4424 non-null  int64
12  Educational special needs                                              4424 non-null  int64
13  Debtor                                                                4424 non-null  int64
14  Tuition fees up to date                                                4424 non-null  int64
15  Gender                                                                4424 non-null  int64
16  Scholarship holder                                                     4424 non-null  int64
17  Age at enrollment                                                      4424 non-null  int64
18  International                                                           4424 non-null  int64
19  Curricular units 1st sem (credited)                                    4424 non-null  int64
20  Curricular units 1st sem (enrolled)                                    4424 non-null  int64
21  Curricular units 1st sem (evaluations)                                4424 non-null  int64
22  Curricular units 1st sem (approved)                                    4424 non-null  int64
23  Curricular units 1st sem (grade)                                        4424 non-null  float64
24  Curricular units 1st sem (without evaluations)                        4424 non-null  int64
25  Curricular units 2nd sem (credited)                                    4424 non-null  int64
26  Curricular units 2nd sem (enrolled)                                    4424 non-null  int64
27  Curricular units 2nd sem (evaluations)                                4424 non-null  int64
28  Curricular units 2nd sem (approved)                                    4424 non-null  int64
29  Curricular units 2nd sem (grade)                                        4424 non-null  float64
30  Curricular units 2nd sem (without evaluations)                        4424 non-null  int64
31  Unemployment rate                                                      4424 non-null  float64
32  Inflation rate                                                         4424 non-null  float64
33  GDP                                                                    4424 non-null  float64
34  Target                                                                4424 non-null  object
dtypes: float64(5), int64(29), object(1)
memory usage: 1.2+ MB
```

Step 3: Target Variable Preprocessing The AI needs a clear, binary goal to predict. We isolated the Target column (which contained the text "Dropout", "Graduate", and "Enrolled") and cleaned it up.

Removing Uncertainty: We filtered out all students labeled "Enrolled" because their final academic outcome is still unknown and would confuse the model.

Converting to Math: Since AI cannot read English words, we mapped the text outcomes into binary numbers: Dropout became 1 and Graduate became 0.

```
# 1. Check what unique text values are currently in the Target column
print("Before cleanup:")
print(df['Target'].value_counts())
print("-" * 30)

# 2. Remove students who are still 'Enrolled' so we only have definitive outcomes
df = df[df['Target'] != 'Enrolled']

# 3. Convert the text to numbers: Dropout becomes 1, Graduate becomes 0
df['Target'] = df['Target'].map({'Dropout': 1, 'Graduate': 0})

# 4. Check the results to make sure it worked
print("After cleanup:")
print(df['Target'].value_counts())
```

```
Before cleanup:
Target
Graduate    2209
Dropout     1421
Enrolled      794
Name: count, dtype: int64
-----
After cleanup:
Target
0      2209
1      1421
Name: count, dtype: int64
```

Step 4: The Train/Test Split If an AI learns on the entire dataset, it might just memorize the answers. We had to separate the data to test its true learning ability.

Separating Features and Target: We split the dataset into X (all 36 columns of student data) and y (the single column containing the 1 or 0 answer key).

The 80/20 Split: Using `train_test_split` from `scikit-learn`, we gave the model 80% of the data to study (train) and hid the remaining 20% to act as a final exam (test).

```
# 1. Import the Machine Learning tools from scikit-learn
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 2. Separate your Features (X) and your Target answer (y)
# X contains all 36 student attributes (grades, age, attendance, etc.)
# y contains ONLY the final answer (1 for Dropout, 0 for Graduate)
X = df.drop(columns=['Target'])
```

```

y = df['Target']

# 3. Split the data: 80% for TRAINING the AI, 20% for TESTING it
# random_state=42 ensures you get the exact same split every time you run it
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, randc

print(f"Training set size: {X_train.shape[0]} students")
print(f"Testing set size: {X_test.shape[0]} students")
print("-" * 30)

# 4. Initialize the AI brain (Random Forest)
model = RandomForestClassifier(n_estimators=100, random_state=42)

# 5. TRAIN THE AI! (This is where the actual learning happens)
print("Training the AI model...")
model.fit(X_train, y_train)
print("Training Complete!")
print("-" * 30)

# 6. Test the AI on the 20% of students it has never seen before
predictions = model.predict(X_test)

# 7. Calculate and print how accurate it was
accuracy = accuracy_score(y_test, predictions)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
print("\nDetailed Performance Report:")
print(classification_report(y_test, predictions, target_names=['Graduate (0)',

```

Training set size: 2904 students

Testing set size: 726 students

Training the AI model...

Training Complete!

Model Accuracy: 89.12%

Detailed Performance Report:

	precision	recall	f1-score	support
Graduate (0)	0.89	0.94	0.91	449
Dropout (1)	0.90	0.81	0.85	277
accuracy			0.89	726
macro avg	0.89	0.88	0.88	726
weighted avg	0.89	0.89	0.89	726

Step 5: Model Training (The "Brain") This is the actual machine learning phase where the algorithm looks for patterns in the student data.

The Algorithm: We selected a RandomForestClassifier. This algorithm works by building hundreds of virtual "decision trees" and having them vote on whether a student will drop

out or graduate based on the patterns it finds.

Learning: We used `model.fit(X_train, y_train)` to force the AI to study the 80% training data and learn exactly which combinations of factors lead to a dropout.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Create the AI "Brain"
# (random_state=42 just keeps the results consistent every time you run it)
model = RandomForestClassifier(random_state=42)

# 2. Train the model using your 80% training data
model.fit(X_train, y_train)

# 3. Give it the 20% test data and ask it to make predictions
y_pred = model.predict(X_test)

# 4. Grade the exam!
accuracy = accuracy_score(y_test, y_pred)
print(f"Final Model Accuracy: {accuracy * 100:.2f}%")
```

Final Model Accuracy: 89.12%

Step 6: Model Evaluation After studying, we tested the model's accuracy on the 20% of data it had never seen before.

Making Predictions: We used `model.predict(X_test)` to have the AI guess the outcomes for the hidden test students.

Grading the AI: We compared the AI's guesses to the actual real-world outcomes using `accuracy_score`. The model successfully predicted the correct outcome roughly 89% of the time, proving it actually learned the underlying patterns rather than just guessing.

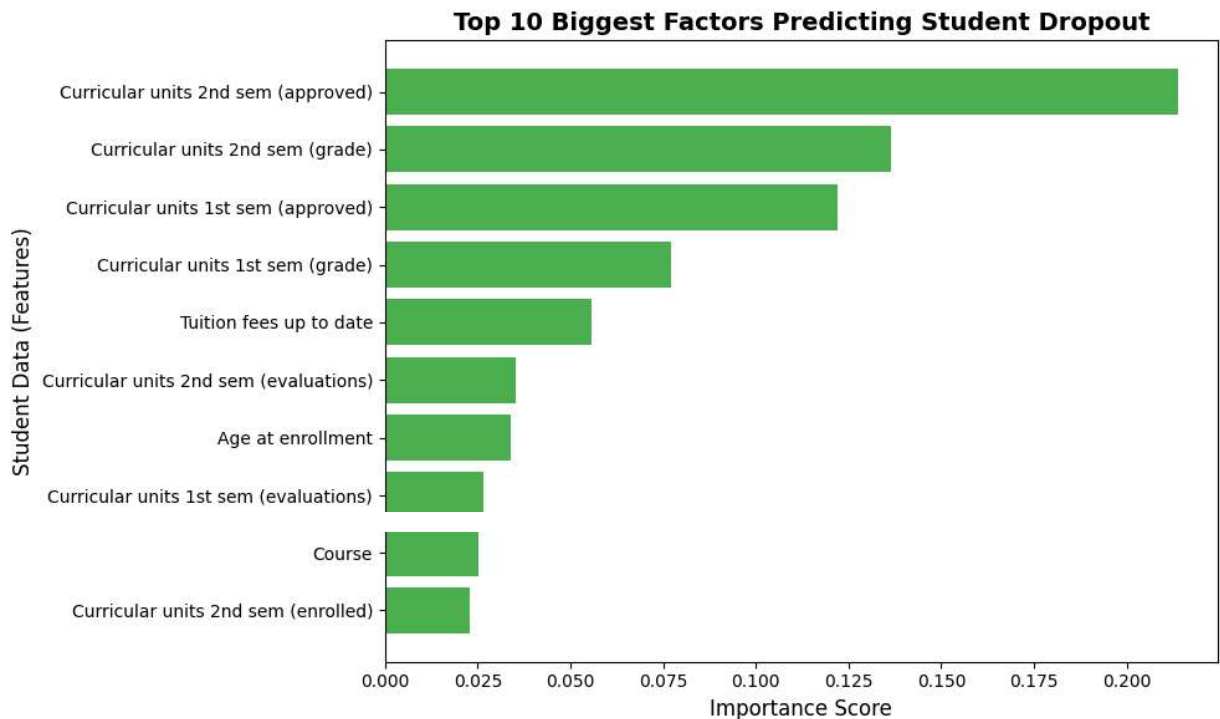
```
import matplotlib.pyplot as plt
import pandas as pd

# 1. Extract the importance scores from our trained model
importances = model.feature_importances_

# 2. Match the scores to the column names and sort them
feature_df = pd.DataFrame({'Feature': X.columns, 'Importance': importances})
feature_df = feature_df.sort_values(by='Importance', ascending=False).head(10)

# 3. Draw the bar chart
plt.figure(figsize=(10, 6))
plt.barh(feature_df['Feature'][:-1], feature_df['Importance'][:-1], color='#4
plt.title('Top 10 Biggest Factors Predicting Student Dropout', fontsize=14, for
```

```
plt.xlabel('Importance Score', fontsize=12)
plt.ylabel('Student Data (Features)', fontsize=12)
plt.tight_layout()
plt.show()
```



Step 7: Feature Importance Extraction Because we need to explain why the AI makes its decisions, we extracted its internal logic.

Extracting the Logic: Random Forests calculate how much they rely on each specific column when making a decision. We pulled these scores using `model.feature_importances_`.

Visualization: We used the matplotlib library to draw a bar chart mapping the top 10 most critical factors. The visual output revealed that passing 1st and 2nd-semester classes, alongside keeping tuition fees up to date, were the most powerful predictors of whether a student would stay or drop out.

This Random Forest model predicts student dropouts with 89% accuracy. Based on the feature importance chart, the administration should focus early interventions on students who fail to get their 1st and 2nd-semester curricular units approved, as academic failure in the first year is the leading indicator of dropping out